

泰坦尼克号学习笔记

一、数据清洗和特征处理方法

1. 原始特征筛选

原始数据集包含：PassengerId、Survived、Pclass、Name、Sex、Age、SibSp、Parch、Ticket、Fare、Cabin、Embarked。删除无预测价值字段：PassengerId（无意义）、Ticket（无意义）、Cabin（船舱缺失过多，大量空值）；Name、SibSp、Parch，这3列用于构造新特征后移除。

2. 构造新特征

familyAll：家庭总人数， $\text{familyAll} = \text{SibSp} + \text{Parch} + 1$ ，代表乘客自身 + 兄弟姐妹 + 父母子女，反映家庭规模，海难中家庭人数会影响生存概率。

identify：从姓名中提取称谓（Mr./Mrs./Miss./Master./Other），提炼社会身份信息，代替完整姓名作为分类特征。

3. 缺失值填充与特征编码 Pipeline+ColumnTransformer

1. 类别特征：Sex、Embarked、identify

- 缺失值：众数填充（`SimpleImputer(strategy="most_frequent")`）
- 编码：OneHot 独热编码，把离散文本转为 0-1 数值，供神经网络输入

2. 数值特征：Pclass、Age、Fare、familyAll

- 缺失值：均值填充（`SimpleImputer(strategy="mean")`）
- 标准化：StandardScaler，将数值缩放到均值 0 方差 1，加速神经网络收敛。

预处理放在 K 折循环内部，每一折只用该折训练集的均值/众数做 fit，验证集只 transform。不能在全部数据集上提前拟合预处理，否则造成数据泄露

二、模型结构、损失函数、优化器及主要超参数

1. 神经网络结构 (PyTorch 全连接网络 Net)

输入层 $\rightarrow$ `Linear(fea, 28)` $\rightarrow$ `BatchNorm1d` $\rightarrow$ ReLU 激活 $\rightarrow$ `Dropout(0.28)`
 $\rightarrow$ `Linear(28, 14)` $\rightarrow$ `BatchNorm1d` $\rightarrow$ ReLU 激活 $\rightarrow$ `Dropout(0.28)`
 $\rightarrow$ `Linear(14, 1)` 输出 logit

- BatchNorm：批量归一化，加速收敛、稳定训练
- Dropout：随机失活，抑制模型过拟合，`dropout=0.28`

2. 损失函数

`nn.BCEWithLogitsLoss()`

3. 优化器

Adam 优化器，学习率 $lr = 0.01$ ，自适应学习率，收敛速度快

4. 主要超参数

- 隐藏层 1 神经元：28；隐藏层 2 神经元：14
- Dropout 概率：0.28
- 学习率：0.01
- Batch size：32
- 最大训练轮数 epoch：60
- 早停 patience=10：连续 10 轮验证损失不下降就提前终止训练，防止过拟合

这些超参数的设定是写到 k 折时试出来比较稳定的，准确率综合下来也还行的

三、数据划分方式、随机种子和实验配置

1. 数据集分层划分

原始全部数据先切分：85% 训练数据集 X_k + 15% 独立 Holdout 测试集 X_{test}

在 85% 的 X_k 训练集上，做 5 折 KFold 交叉验证，shuffle 打乱，random_state=42。5 折作用：评估网络架构是否稳定，得到 5 组验证准确率，计算均值与标准差，判断模型稳定性。

训练最终模型时：将 X_k 再拆分为 80% 训练 / 20% 验证集，验证集用于早停。

2. 随机种子固定

都是 42

3. 实验配置

(1) 模型结构

搭建 2 层全连接神经网络：输入层 $\rightarrow$ 隐藏层 1 (28 个神经元) $\rightarrow$ 隐藏层 2 (14 个神经元) $\rightarrow$ 输出层 1 个神经元；隐藏层使用激活函数，添加 Dropout 层（丢弃概率 0.28）做正则化。任务为二分类，输出使用 BCEWithLogitsLoss。

(2) 训练超参数

优化器：Adam 学习率 $lr = 0.01$ 最大训练轮数 epoch：60

损失函数: `nn.BCEWithLogitsLoss()`

(3) 软件与硬件环境

编程语言: Python

深度学习框架: PyTorch

依赖库: pandas、numpy、scikit-learn、matplotlib

运行环境: CPU

四、训练过程中遇到的问题及解决方法

1. 数据泄露

(1) 最开始尝试在 K 折循环外, 对全部 X_k 一次性 fit 预处理, 但是必须分了层, 才能拿到训练集单独的均值

解决: 使用 `ColumnTransformer + Pipeline`, 预处理封装在循环内; 每一折, 仅用当前折训练集 fit, 验证集只用 transform, 不学习验证集信息。

(2) 实不相瞒, 最开始的时候, 我进入了迷之癫狂状态, 试图找出测试集最高的准确率, 直接改到了数据集的随机种子身上, 成功找出了把测试集划分的最简单的随机种子。笑死我了, 但这样做根本没有任何意义, 回头一看我的训练集和验证集的准确率低且相差很大

测试集的准确率实际上并不代表着我这个模型如何!! 要综合和多个指标来看, 我只是在不断试出最利于准确率的数据划分方式而已, 从某种方面上来说, 我这样不断看结果然后再修改怎么不算一种数据泄露呢

2. 传网络的时候特征数对不上

(1) 忘记读热编码之后一个特征会变成几个 解决: 加上或者搞成自动计算

3. PyTorch 标签 y 与模型输出张量维度不一致, 计算损失时报错

标签 y 读取出来是一维张量 $[n]$, 而神经网络输出 `logit` 是二维张量 $[n, 1]$, 二者形状不匹配, `BCEWithLogitsLoss` 计算时报错。

解决方法: 在把标签转为 torch 张量后, 使用 `.unsqueeze(1)`, 将一维标签扩展为二维

4. 混淆多随机种子重复单次划分和 K 折交叉验证

想通过循环更换随机种子，看能否观察出最佳的超参数和网络设置

多次换种子单次划分：每次验证集是随机抽的子集，样本可能重复、部分样本永远不会进入验证集，评估存在偏差，小数据集容易运气成分大。

K 折交叉验证：把训练集切分为 K 份，每一个样本都会恰好有一次作为验证集，没有遗漏、没有重复，对小数据集评估结果更加可靠，能更真实反映模型泛化能力。

五、测试集准确率、实验结果分析和心得体会

1. 实验结果

- 5 折交叉验证平均准确率：0.8256 5 折准确率标准差：0.0191
- Holdout 独立测试集准确率：0.8284

2. 结果分析

(1) 5 折交叉验证结果分析

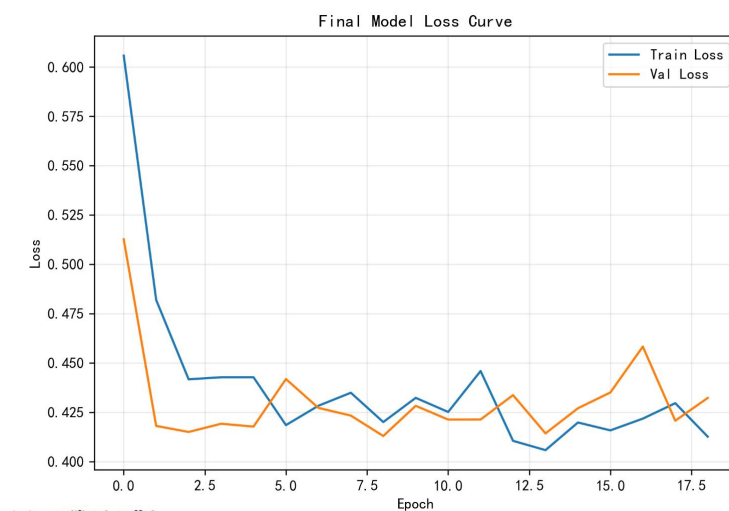
5 折平均准确率 0.8256，标准差仅 0.0191。各折准确率最低 0.8013、最高 0.8553，整体波动很小，说明当前这套网络结构与超参数的训练稳定性较好

(2) 最终模型准确率曲线

测试集的准确率落在训练、验证准确率区间中间，没有出现验证集和测试集差距巨大的情况，没有过拟合，也挺合理的

(3) 最终模型损失曲线

训练损失与验证损失在训练初期快速下降，模型快速收敛，训练中后期两条损失曲线出现小幅震荡，没有出现验证损失持续明显上升的过拟合现象



训练集与验证集损失

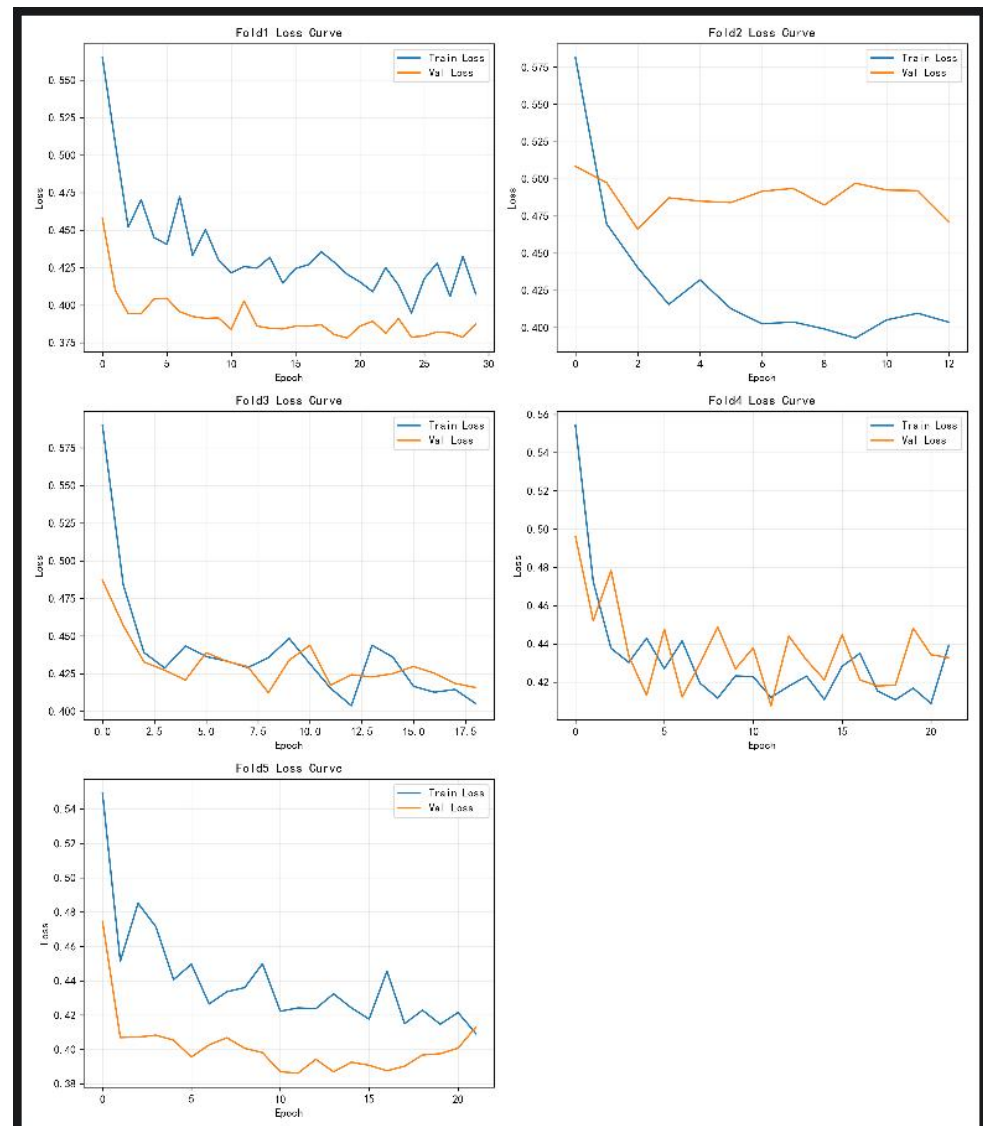
(4) 5 折分折 Loss 子图

Fold3、Fold4 曲线表现正常

Fold1、Fold5 出现训练损失高于验证损失的现象。可能是数据集样本量小，折划分之后子集分布不均衡：这两折的验证集样本整体更容易预测，验证集天然损失更低，训练集中难预测样本占比更高。

Fold2 存在轻微过拟合趋势：训练损失持续下降，但是验证损失在初期下降后不再降低，后期缓慢上升。说明在该折的数据子集上，模型在训练轮次增加后，开始学习训练集的特有噪声，泛化能力有所衰减。

虽然各折训练曲线形态存在差异，但从量化指标来看，5 折验证集平均准确率 0.8256，标准差仅 0.0191。各折准确率没有出现极端暴跌，应该只是部分子集上出现分布偏移或轻微过拟合



(5) 整体结论

Holdout 独立测试集准确率 0.8284，和 5 折交叉验证平均准确率 0.8256 非常接近，两者几乎持平。K 折评估得到的性能可以代表模型在未知新数据上的泛化能力，K 折的评估结果可靠

3. 心得体会

之前写传统机器学习方法时并没有进行特征工程，后面看到有帖子说特征工程是很重要的一部分，我追悔莫及，于是这次加上了特征工程。这次的实验对 k 折和初步深度学习的理解都有很大的帮助，会有很多细节上的问题并不是最初想的那么简单。不过好像还是随机森林的效果更好，这个数据太少了。